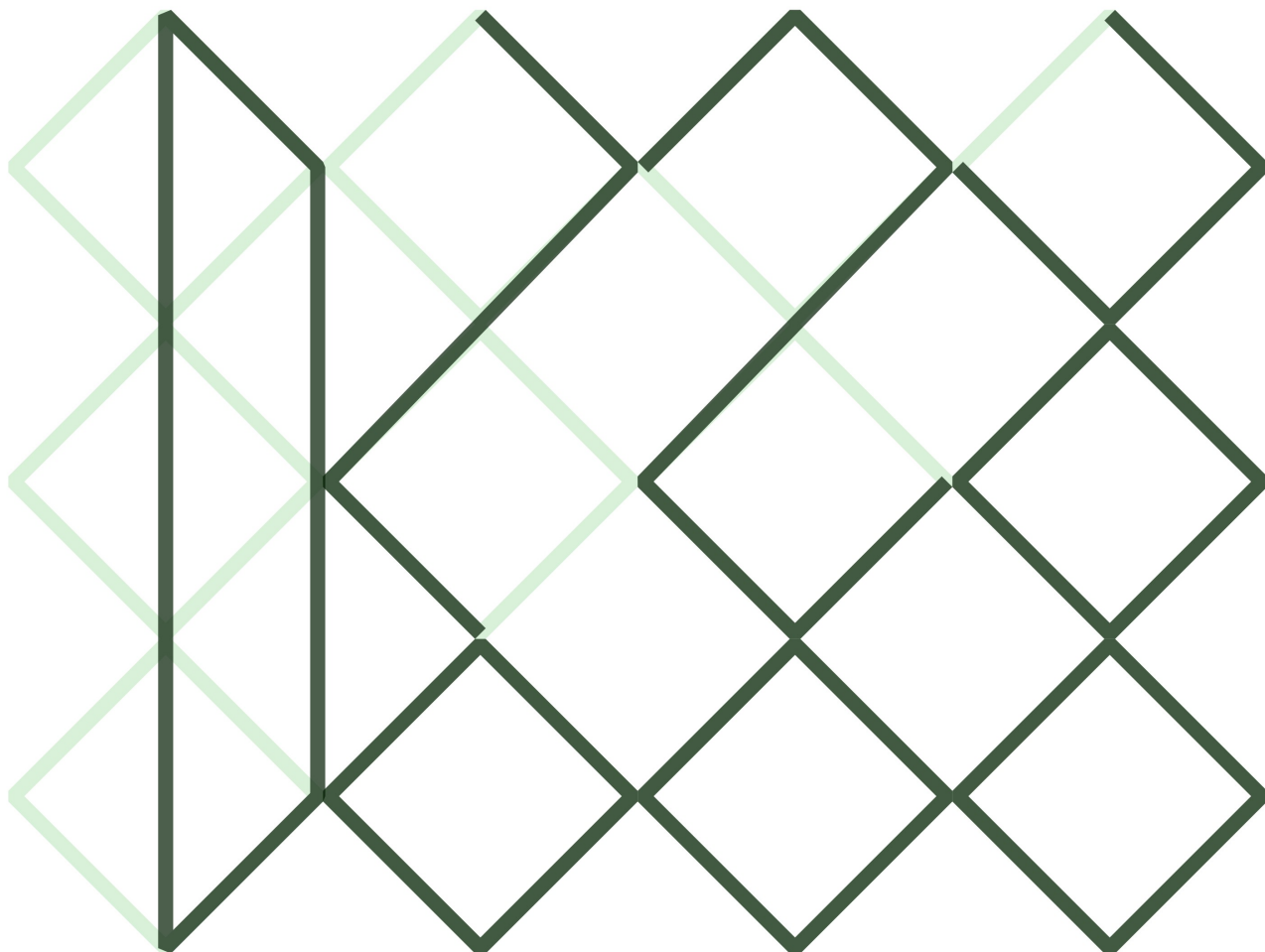
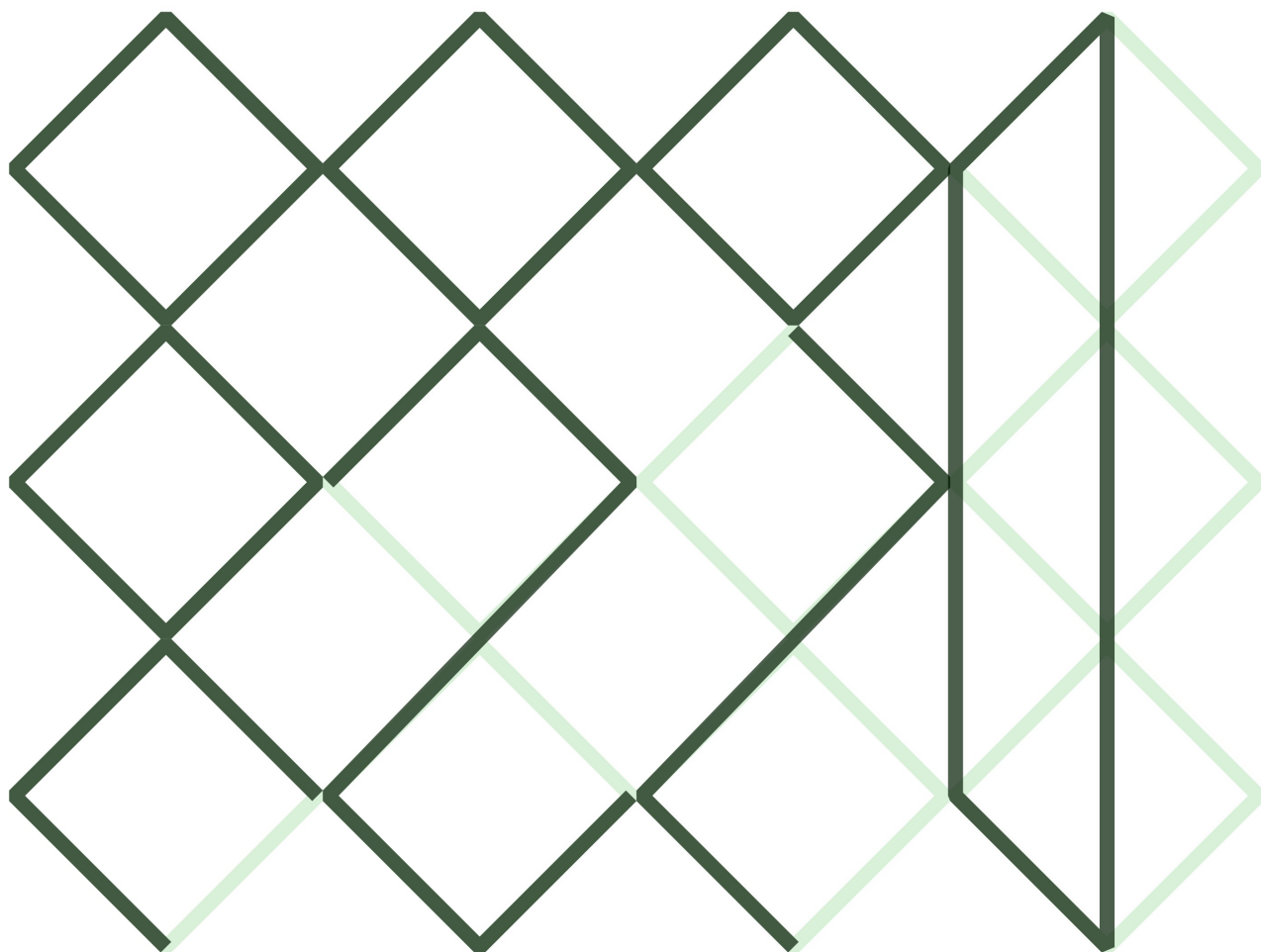




D - L a n g
2 0 2 6

語 言 D
6 2 0 2



目次

目次	1
ArchLinux と Hyprland で理想の環境へ	2
J 科実験難易度ティア表作ってみた.....	9
プログラムを連続実行させよう	18
プロンプトエンジニアリング実践してみた	24
ローカルリポジトリを複数台で同期したい件.....	29
あとがき	42

ArchLinux と Hyprland で理想の環境へ

著者：Nanmo

Nanmo 君はいままで Windows をメインで使い続け、Linux にはほとんど触れてきませんでした。でも、Nanmo 君は新しいものに挑戦してみたいと思っています。そんな折、某 IT 勉強会で Linux の話を聞き、ついに Linux を使うきっかけができました。せっかくなら上級者向けの ArchLinux と、見た目がイケてる Hyprland を使いたい！そんなわけで、ArchLinux+Hyprland の環境構築記です。

私は初心者であるため、一部正確性にかける文言や、間違った情報があるかもしれませんが、ご容赦ください。

1 ArchLinux, Hyprland って何

1.1 ArchLinux

ArchLinux とは独自に開発された非常にシンプルな Linux ディストリビューション (色々組み合わせて使いやすくまとめたもの) の一つです [1]。ArchLinux はシンプルであることを原則の一つとしているため、Ubuntu などのように最初からデスクトップ環境 (GUI) は用意されておらず、すべて自分でコマンドライン (黒い画面) から導入する必要があります [1]。ただ、その性質からシステムの全容が理解しやすかったり、最小限の状態から自分好みにカスタマイズしていけるなどの利点があります。

私はこのカスタマイズ性に惹かれて導入しました。

1.2 Hyprland

Hyprland は Wayland という規格に合わせて作られた、各アプリが描画したものを管理してモニターに表示するソフトウェア (Wayland コンポジタ) です [2]。大きな特徴は Windows のようにウィンドウを自由に重ねたり移動させられるフローティング型で

はなく、画面を分割して重なることなく規則的に配置するタイル型を主に採用していることです。これにより、画面の無駄なスペースを減らすことができ、この特性上キーボードでの画面の切り替えやフォーカスの移動がとてもやりやすいです。それに加え、カスタマイズ性がとても高いです。これらの特徴を兼ね備えたコンポジタは他にもあるのですが、とりあえず今回は Hyprland を使うこととします。何はともあれ Hyprland の公式サイト [3] を見てほしいです！ Hyprland の魅力と美しさが存分に伝わってきます！

2 ArchLinux の導入

では、ArchLinux を導入していきます。詳しい方法は、ArchWiki のインストールガイド [4] に書いてあるので、ここではざっくりとやることを書いていきます。

適当な CD や DVD、USB メモリに公式で配布されている ISO ファイルをそれぞれに対応した書き込みソフトを用いて書き込みます。これを導入対象の PC に入れて、ブートするとライブ環境に移行するので、そこで設定していきます。

ライブ環境が起動できたら、キーボードレイアウト、インターネット接続の確認、システムクロックの設定などを行います。その後、インストールに使用するディスクに対して、パーティション (ストレージ上の区画) の作成・割り当て、フォーマット、ファイルシステムのマウント (パーティション内のファイルシステムを特定のディレクトリから使えるようにする) をします。

ミラー (ダウンロードするサーバー) を選択して、base パッケージ、Linux カーネル、ファームウェアをインストールします。

あとは、fstab の生成、arch-chroot (インストール先への移行)、タイムゾーンの設定、ローカリゼーション (地域, 言語, 文字コードなどの設定)、ネットワーク設定、root パスワードの設定、好きなブートローダーを入れて再起動すれば ArchLinux の導入完了で

す！ArchLinux を再起動したらお好みで他のパッケージ、アプリをいれるなどして使いやすくしましょう！

3 Hyprland の導入

次に、Hyprland を導入していきます。と言ってもとても簡単で、pacman(ArchLinux のパッケージマネージャ) を使って `sudo pacman -S hyprland` をするだけです。詳しくは HyprlandWiki の [installation\[6\]](#) を見てください。

4 導入後

Hyprland を何も入れず初期設定で起動するとこんな画面になります。(実際はチュートリアルも表示される)



図 1: Hyprland 初期画面

標準の壁紙のみのシンプルな画面になっていることがわかんと思います。Hyprland はあくまでもコンポジタであり、その他のターミナルやブラウザなどのアプリケーションは自分でいれる必

必要があります。特に、Hyprland ではほぼ必須とされるアプリケーションが指定されているため、チュートリアルや HyprlandWiki の Musthave[7] を参考にして導入します。

ここまでやったら、やるべきことはほぼ終わりです。基本的にはどのアプリも .conf ファイルや .lua ファイルなどの設定ファイルを有しているので、各 wiki などを見つつ設定します。ここが一番時間がかかるし楽しいところです。頑張って自分好みの環境を創り上げましょう！

5 まとめ

ほぼ初めての Linux の導入、さらに ArchLinux ということでもかなり身構えていたのですが、やってみると各 Wiki に情報が詳細に記述しており、意外と簡単に導入できてしまいました。どちらかというと導入後の方が音声やキーリング、設定ファイルなどで手間取っています。私の現在地もまだ導入してアプリを入れ始めたくらいで、まだまだ「とりあえず使える」の域を出ていません。もともと HyprlandWiki や Reddit に挙がっているカスタマイズを見てこの構成にしたので、少しずつ育てていきたいなと思います。ちなみに今の私の画面はこんな感じです。

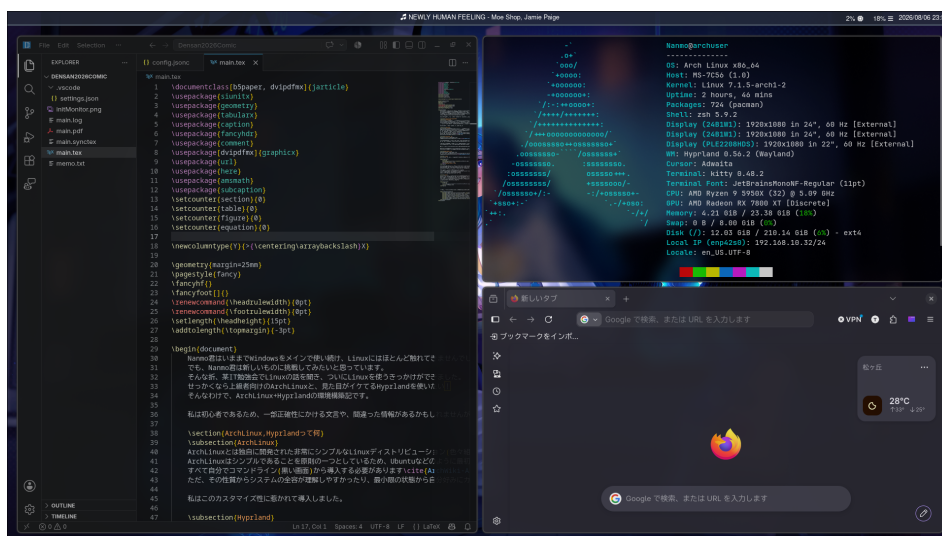


図 2: 現時点での画面

ぜひあなたも ArchLinux と Hyprland を導入し最高の環境を創り上げましょう！

参考文献

- [1] ArchLinux-ArchWiki,https://wiki.archlinux.jp/index.php/Arch_Linux
- [2] Hyprland-ArchWiki,<https://wiki.archlinux.jp/index.php/Hyprland>
- [3] Hyprland,<https://hypr.land/>
- [4] インストールガイド,<https://wiki.archlinux.jp/index.php/インストールガイド>
- [5] ArchWiki,<https://wiki.archlinux.jp>
- [6] Installation-Hyprland,<https://wiki.hypr.land/Getting-Started/Installation/>

- [7] Musthave-Hyprland,<https://wiki.hypr.land/Useful-Utilities/Must-have/>

J 科実験難易度ティア表作ってみた

著者：ZIMMAD

〇はじめに

みなさんこんにちは.ZIMMAD です.今回の部誌では,JavaScript を使って何か作りたいと思っていましたが,試験期間とかぶってしまったため断念しました.そこで,その代わりに(?),1~4 年生前期の実験実習が無事に終了したので,「電子情報工学科実験実習難易度 Tier 表」を作って,それを記事にしようと思います.

〇判断基準・表の形式など

今回は 4 年生までの実験実習について,以下の項目で難易度を判断し,S~D(一番上の難易度が S)までのランクに振り分けて表を作ります.なお,以下の基準で表を作った場合,1 年生の実験実習が一番下の D ランクに固まりやすいと考えられるため,今回は 1 年生の実験実習は省きます(今回の表の実験内容は自分が経験したもののみとします.もしかしたら,自分と学年の違う方は,なくなっている実験課題があったり,内容が変わっている実験課題があるかもしれません.).

- ・**実験の難易度** : 実験課題自体の難しさ,学年に対してどのくらいレベルが高い(授業で習うのは実験後になる,授業で扱わず,内容が発展的など)ことを要求されるかなど.また,自分で一から調べて実験を行わなければいけないのか,指導書やテーマ説明でどうにかなるものなのかについても考える.
- ・**レポートの難易度** : 1 つのレポートに含まれる表やグラフ,図の数,考察課題など,多ければ多いほど難易度は上がるためこれも加味する.
- ・**実験時間** : 明らかに 1 週実験では時間が足りないもの,場合によっては 2 週あっても時間が足りないと感じるものはそうでないものと比べて難易度が高いと判断する.

表の形式は以下の図のようにします.

S	
A	
B	
C	
D	

図の通り、難易度が高い順に S,A,B,C,D となります。それでは、2J の実験から振り分けていきましょう。

○2J 実験実習

2J の実験は、

1. **2J 前期** ダイオードの特性 プレゼンテーション入門 ホイトストンブリッジによる抵抗測定
最小二乗法によるデータ解析 多倍長演算 直流電源の負荷特性 論理ゲートの作成
2. **2J 後期** トランジスタの静特性 マイコン(1) -ステップモータ- マイコン(2) -LCD 制御- 交流回路の基礎(1) 基本ソーティングアルゴリズム 高精度演算

となります。まず、2 年前期の実験について見ていきましょう。

・**ダイオードの特性**:ダイオードの順方向特性と逆方向特性を測定する実験、実験項目は結構多いのですが、回路の組み換えは、ダイオードの向きを 1 度変えるだけなので、そこまで難しくはないかと思います。レポートについては、標準的なハードウェア系実験の分量で、まあ大丈夫でしょう。よって、C としたいと思います。

・**プレゼンテーション入門**:スライドを作成して、担当教員からアドバイスを貰い、修正する実験。プレレポートとして表紙とアウトラインの提出を求められる(逆に言うとプレはそれだけ)。本レポートはそれに作成したスライドと考察を追加するだけ。2J 実験だと一番楽かも。よって D に置いときましょうか。

・**ホイトストンブリッジによる抵抗特性**:これも 2 年生の実験実習としては妥当な難易度。レポートも特に言うこと無し。C でいいでしょう。

・**最小二乗法**:最小二乗法を使って,後述する直流電源の負荷特性の実験のデータを解析します.ここに来て初めて,学年に対して発展的な内容となっていますが(最小二乗法をしっかり授業でやるのは3年生の数値解析),たいして難しくない内容で,実験課題の内容も2つだけ,考察課題も3つだけなので,これはDでもいいんじゃないかなと思います.

・**多倍長演算**:C言語のint型のサイズでは扱えないような大きい整数の足し算,引き算,割り算を行います.多分,2J前期の実験の最難関とっていいんじゃないかなと思います.課題の量やレポートに書く量もまあまあ多いです.ただ,2J前期実験の中でこいつだけ2週間なんですよね.自分も特に実験時間外で遅くまで進めることはしなかったように思います.あと,原理に基本的な考え方が書いてあったり,作る部分は多倍長整数の入力,加算,減算,除算の関数と,動作確認用のmain関数のみなので,あまりにも難易度が高いわけではなく,Sではないだろうと思います.よってAくらいが妥当かなと考えました.自分よりプログラミングが得意な人にとってはBくらいに感じるかも.

・**直流電源の負荷特性**:実験内容とレポートはたいしたことないと思うのですが,計算に2J後期で習うテブナンの定理を使用する必要があり,Bくらいが妥当でしょう.

・**論理ゲートの作成**:作成する回路自体はそこまで難しいものではないのですが,2年生のうちはまだブレッドボードに慣れていないと考えられる(というか自分がそうだった)ので,Cくらいでいいかなと思います.ブレッドボードが苦手なので個人的にはBくらいでもいいと思うのですが(指が太いんで細かいのはピンセットがないと厳しいんです.),こんな感じでも一人で回路作成を担当して正しく回路が作れたため,Cにしておこうと思います.

さて,2J後期も見えていきましょう.

・**トランジスタの静特性**:実験中はグラフ係の人が忙しいかも(グラフ3つのうちの1つは5本の曲線が1つのグラフに入ります.多少器用さがいるかもです).ただ,それ以外は普通なんでCで.

・**マイコン(1)-ステップモーター**:指導書をよく読んでプログラムの穴埋めをやって,モーターを回せば実験課題はおしまい.レポートも分量は多くないので,これはDでいいでしょう.

・**マイコン(2)-LCD制御**:LCDボードで文字を表示する実験.これもコードのひな形があるため楽.Dでいいでしょう.

・**交流回路の基礎(1)**:記念すべき(?)群馬高専電子情報工学科実験実習でのオシロスコープ初登場.2週実験といっても1回目はオシロスコープの使い方,2回目は測定という形なので,実験課題,レポートの量もそこまで多いわけではありませんが,あんな1回ぽっちでオシロスコープの使い方をマスターするわけがないので,Bとしておきます.高難易度というわけではないですが,オシロスコープの使い方は覚

えておかないとこの先の実験で詰む可能性があります。

・**基本ソーティングアルゴリズム**: バブルソート,挿入ソート,選択ソートのプログラムを作成します.人によってはコマンドライン上でプログラムを実行する部分が少し慣れないかと思われますが,方法が指導書に書いてあるんで大丈夫でしょう.よって D で問題ないかと思います.

・**高精度演算**: 円周率を近似します.その時に多倍長整数を使うのですが,正しく動いていれば作成するプログラムは比較的難しくないかと思います.多倍長整数を乗り越えてきた学生がやる課題(ただし前回のプログラムが正しく動作しないと詰み)と考えれば B くらいが妥当ですかね.

ここまでが 2 年生の実験になります.途中経過の表は以下のようになります.

S	
A	多倍長整数,
B	交流回路の基礎(1),高精度演算,直流電源の負荷特性
C	トランジスタの静特性,論理ゲートの作成,ホイートストンブリッジによる抵抗特性,ダイオードの特性
D	基本ソーティングアルゴリズム,最小二乗法,マイコン(1)-ステップモーター,マイコン(2)-LCD制御,プレゼンテーション入門

○3J 実験実習

以下が 3J の実験実習です.

1. **3J 前期** トランジスタの増幅特性 マイコン(3) -シリアル通信- 交流回路の基礎(2) AI プログラミング入門 Tex 基礎実習 生成 AI 基礎実習
2. **3J 後期** LC フィルタの特性 トランジスタの増幅回路の設計製作 デジタル IC の使い方 デジタル IC を使った論理回路の設計と実装 VMware によるネットワーク環境実習 UNIX 環境の構築と CUI 環境

はい.魔境です.面倒なのでいっしょに見ていくと,

・トランジスタの増幅特性,LC フィルタの特性,トランジスタの増幅回路の設計製作:このあたりからハードウェアの実験も時間がかかり始める.回路も複雑になるし,オシロスコープが当然のように登場する.先ほど,ハードウェアの実験を C にしていたことが多いため,これらは B くらいの難易度はあると思われる.

・交流回路の基礎(2):実験自体はたいしたことないのですが,問題は実験の準備にあり,2 年の交流回路(1)の実験を参考にして自分たちで実験を考える必要がありました.かなり焦った記憶があります.これは A でいいかな.

・Tex 基礎実習:レポートの作成等に使うワープロソフト LaTeX の使い方を学ぶ...と思いきやほぼ自分で調べます.やること自体は難しくありませんが,何を調べていいかわからない状態に陥る可能性があるため,B くらいの難易度にしておこうと思います.

・生成 AI 基礎実習:今話題の生成 AI にいろいろやらせる.文書やコードを生成したり,模擬面接をしたり...など.実験自体はめちゃくちゃ楽(先生公認).ただしレポートの記述量が多く,word を使うことを指定されているため,word が苦手な自分には苦行だった.よって C かな.

・AI プログラミング入門:python でのプログラム作成と CNN のサンプルプログラムの実行.割と簡単.D くらいの難易度が妥当.

・デジタル IC の使い方:デジタル IC を使用して回路を作り,チャタリングを観測 & チャタリング除去回路の効果を確認する実験.後述のデジタル IC を使った論理回路の設計と実装の予備知識みたいなもんです.チャタリングをちゃんと観測できれば割と早く終わります.オシロスコープも出てきます.難易度は B くらいだと思います.

・デジタル IC を使った論理回路の設計と実装:誰に何を言われてもこれだけは S にさせてください.実験内容はというと,ワイヤラッピングでデジタル IC の部品を使ったカウンタを設計・実装するというもの.ワイヤラッピングは,部品をはんだ付けでくっつけるのではなく,専用の器具を使って,被覆を剥いた銅線を巻き付けて,回路の配線を行う方法です.まずこれに時間がかかります.3 人くらいのグループでやるのですが,もたもたしていると 2 週でも足りないかもしれません.さらに,ラッピングを行う前に,設計を行うのですがその時使う設計ソフトの操作に若干のクセがあり,やりづらいです.レポートも決して楽でないため,これは S でいいと思います.回路系では最高難易度では?

・VMware によるネットワーク環境実習:絶対に S にしなければならない実験その 2.まず実験課題,レポートに書くことの量がめちゃくちゃ多いです.多倍長がかわいく見えてくるレベルです.仮想環境の使い方を学ぶための実験なのですが,実験課題の中には,ネットワーク構成図を描いたり html,css,javascript を使ってウェブページを作る実験もありました.ちなみにこれ,一から全て調べる必要があります.指導書には VMware についての概要と,起動方法くらいしか書いてありません.自分は電算

部でその当時はウェブ班に入っていたため、調べるのが少し減ったのですが、とにかくレポートが大変でした。自分の場合は、冬休みが入り、学校のPCが使えない日があったこともあり、レポートがぎりぎりになってしまい提出日には軽く寝不足でした。絶対にもう二度とやりたくねえ。

・**UNIX 環境の構築と CUI 環境:**仮想環境下で認識度のプログラムをコマンドライン上で実行する実験です。シェルスクリプトの書き方等は自分で調べる必要がありますが、ほとんど自動でやってくれることもあり、楽でした。D でいいと思います。

さて、3 年の実験実習を入れた表はこんな感じです。

S	VMwareによるネットワーク環境実習, デジタルICを使った論理回路の設計と実装
A	多倍長整数, 交流回路の基礎(2),
B	トランジスタの増幅特性, LCフィルタの特性, トランジスタの増幅回路, 交流回路の基礎(1), 高精度演算, Tex基礎実習, 直流電源の負荷特性
C	トランジスタの静特性, 論理ゲートの作成, ホイートストンブリッジによる抵抗特性, ダイオードの特性, 生成AI基礎実習
D	基本ソーティングアルゴリズム, 最小二乗法, マイコン(1)-ステップモーター, マイコン(2)-LCD制御, AIプログラミング入門, プレゼンテーション入門

〇4J 実験実習～そして表の完成へ～

最後,4Jの実験は,

SPICE 実習 OP アンプの特性 デジタル回路設計と製作 公開鍵暗号

の4テーマです.全て2週実験です.

SPICE 実習は,LTspice という電子回路シミュレーションソフトを使って,回路のシミュレーションを行う実験です.かなり簡単だったので,Dでいいかと思います.回路も5個くらいだったと思うので,時間もそんなにかかりません.正直これに2週もいらないう.

OP アンプは2週実験ですが,1週で終わるくらいの内容です.レポートは図が多いこと以外は普通です.これはCくらいが妥当だと思います.

デジタル回路はキッチンタイマを論理回路の実習ボードを使用して設計する課題です.内容は少し面倒ですが,2週あるので何とか終わるか.これだけ聞くとBくらいかなと予想する方が多いと思われますが,この実験の恐ろしさは以下の2点にあります.

その1:実験担当教職員に動作確認をしてもらう必要があり,それが終わらないとレポートが書けない(レポートに添付する動作確認シートに担当教員の印が必要).

その2:2週目の実験の終わりまでに動作確認が終えられない場合,後日担当教員が指定する方法で動作確認をしてもらう必要がある.

はい.放課後に残ってやるのは良いにしても,終わったらすぐ終了ではなく,教員に連絡を取る必要があります.それで間違っていればそこでやり直しです.とはいえ,レポートは難しくないでこの表だとAくらいの難易度だと思います.

公開鍵暗号:間違いなくSです.大事なことのでもう一度.この実験の難易度は間違いなくSです.公開鍵暗号の鍵生成,暗号化,復号のプログラムを作る実験ですが,まずこれらに使うアルゴリズムは繰り返し2乗法以外は全て調べる必要があります.これだけなら,プログラムの難易度的にAくらいでいいと思っていた時期が私にもありました.しかし,こいつの恐ろしいところは他にあり,それはレポートの量.特に実験結果.各主要関数のフローチャート,外部設計・内部設計,複雑度なども結果とともに記述する必要があります.考察課題もいくつかあり, レポートは想像を絶するほど大変です.よって,これもSとしたいと思います.

さて,完成した表は以下ようになります.

S	VMwareによるネットワーク環境実習,公開鍵暗号,デジタルICを使った論理回路の設計と実装
A	デジタル回路設計と製作,多倍長整数,交流回路の基礎(2)
B	トランジスタの増幅特性,LCフィルタの特性,トランジスタの増幅回路,交流回路の基礎(1),高精度演算, ,Tex基礎実習,直流電源の負荷特性
C	OPアンプの特性,トランジスタの静特性,論理ゲートの作成,ホイートストンブリッジによる抵抗特性,ダイオードの特性,生成AI基礎実習
D	基本ソートングアルゴリズム,最小二乗法,マイコン(1)-ステップモーター,マイコン(2)-LCD制御, AIプログラミング入門,spice実習,プレゼンテーション入門

〇まとめ

今回は電子情報工学科実験実習の難易度 Tier 表を作ってみました.いかがだったでしょうか?個人的には,C,D あたりに多くの実験が固まってしまったなと感じました.最後に,これは自分の独断と偏見です.あまり真に受けすぎないように,でも少しだけ参考にしてもらえると幸いです.

プログラムを連続実行させよう

著者：ゆーてぃー

1 はじめに

こんにちは, ゆーてぃーです. 今回はプログラムを連続実行させてみようと思います.

連続実行ができると, 多くのデータを一度に処理できます. また, 仕事ができる人のように見えるかもしれません (自己満).

2 必要なもの

- WSL(Ubuntu) などシェルスクリプトが実行できる環境
- テキストエディタ (VSCode)
- C 言語のプログラムをコンパイルできる環境 (VSCode)

3 実際にやってみる

今回は, 学生のテストの点数のデータを与え, 点数の平均値, 最大値, 最小値を計算してみます.

3.1 WSL(Ubuntu) の導入

WSL(Ubuntu) の導入には以下の動画を見て導入しました. シェルスクリプト (.sh) が実行できる環境であれば WSL 以外でも構いません.

バージョン:Ubuntu 24.04.4 LTS

(GNU/Linux 5.15.167.4-microsoft-standard-WSL2 x86_64)



競技プログラミングの環境構築 [VSCode+WSL+AtCoder Library]

【ゆっくり解説】 <https://www.youtube.com/watch?v=uhnASau7fB4>

3.2 プログラムを書く

プログラムをリスト 1 に示します. コメント文を見ればわかると思いますが, プログラムは入力部, 計算部, 出力部に分かれています. 入力部ではファイル入力を行う `fscanf` 関数を使用しました.

プログラムが書けたらコンパイルしておきます.

コンパイル方法 —

```
gcc calc.c -o calc.exe
```

実行方法 —

```
./calc 入力ファイルのパス
```

```
1 #include<stdio.h>
2 int main(int argc,char *argv[]){
3     FILE *fp;
4     int N; //学生数 N を定義
5     double score[101]; //学生のテストの点数のデータを格納する配列
6
7     fp=fopen(argv[1],"r"); //ファイルを開く
8     if(fp==NULL){ //開けなかった場合の処理
9         printf("failed.\n");
10        return -1;
11    }
12    fscanf(fp,"%d",&N); //学生数 N を入力
13    for(int i=0;i<N;i++){
14        fscanf(fp,"%lf",&score[i]); //学生のテストの点数を入力
15    }
16    fclose(fp); //ファイルを閉じる
17
18    //平均値, 最小値, 最大値を定義
19    double score_ave=0,score_min=100,score_max=0;
20    for(int i=0;i<N;i++){
21        score_ave+=score[i]; //学生のテストの点数を加算
22        if(score_min>score[i]) score_min=score[i]; //最小値更新
23        if(score_max<score[i]) score_max=score[i]; //最大値更新
24    }
25    score_ave/=N; //学生のテストの点数の総和を人数で割る
26
27    printf("ave: %.2lf\n",score_ave); //平均値出力
28    printf("min: %.2lf\n",score_min); //最小値出力
29    printf("max: %.2lf\n",score_max); //最大値出力
30    return 0;
31 }
```

リスト 1: 平均値, 最大値, 最小値を計算するプログラム

3.3 入力ファイルの準備

プログラムが入力を行えるように, 以下の形式で入力ファイルにデータを書いていきます. N はデータ数, $score_i$ は各学生のテストの点数です.

N

score1 score2 ... score N

3.4 シェルスクリプトを書く

シェルスクリプトをリスト 2 に示します.

echo は出力コマンドです. ファイル名や区切り線を出力します.

for 文で, フォルダ内の各ファイルを順に入力して与えていきます. 各ファイル名が $\{\text{filename}\}$ の部分に当てはまります. これで連続実行をすることができます.

```
1 for filename in input/*.txt
2 do
3     echo "-----"
4     echo ${filename}
5     ./calc.exe ${filename}
6     echo "-----"
7 done
```

リスト 2: シェルスクリプト

3.5 実行

プログラムをコンパイルし, 入力ファイルの準備が終わったら, 実行します.

実行方法

`./run.sh`

```
ut@TomokiPC:~/R8bushi$ ./run.sh
```

```
-----  
input/test01.txt  
ave : 57.98  
min : 0.00  
max : 100.00  
-----
```

```
-----  
input/test02.txt  
ave : 50.37  
min : 0.00  
max : 100.00  
-----
```

```
-----  
input/test03.txt  
ave : 49.02  
min : 1.00  
max : 96.00  
-----
```

```
-----  
input/test04.txt  
ave : 47.38  
min : 0.00  
max : 100.00  
-----
```

実行結果 (一部)

各データごとに平均値, 最小値, 最大値を計算し出力することができました.

4 おわりに

今回はプログラムの連続実行をしてみました. 手作業でデータ 1 つ 1 つを処理するよりも圧倒的に早いので, ぜひやってみてください.

プロンプトエンジニアリング実践してみた

著者：Shimpei Moribe

1. 初めに

私がこの文章を書き始めたのは部誌の締め切りから 2 日後のことである。当然まだ何について書くかさえ決めていなかった。そこで私は、以前から気になっていたプロンプトエンジニアリングを実践し、プロンプトエンジニアリングの効果を検証し、その結果について記すことにした。

2. プロンプトエンジニアリングについて

プロンプトエンジニアリングとは AI に対する指示文(プロンプト)を工夫して、求めている出力を正確に引き出す技術のことである。またプロンプトエンジニアリングの基本原則は、主に以下の 5 つである。

- ① AI に役割を与える 例：「あなたは〇〇の専門家です。」
- ② 具体的な出力形式を与える 例：「箇条書きで」「表形式で」
- ③ 背景情報を提供する 例：「〇〇のために」「〇〇に使うから」
- ④ 制約条件を明記する 例：「〇〇字以内で」「〇〇語で」
- ⑤ 例を示す 例：「以下に示す形式で〇〇して……」

3. 検証方法について

本検証では AI に同じテーマについての文章を何も具体的な指示を与えない場合と、プロンプトエンジニアリングを実践した場合の二通り書かせ、どのような違いが表れるか観察する。また本検証では Copilot を用いる。

また AI には三平方の定理について説明してもらうことにした。

4. 具体的な指示を与えない場合

プロンプト： 三平方の定理の説明をして

AI からの回答：

三平方の定理（ピタゴラスの定理）は、**直角三角形の 3 つの辺の長さの関係を表す数学の定理**だよ。結論をひと言で言うと、

直角をはさむ 2 辺の長さをそれぞれ 2 乗して足すと、斜辺の長さの 2 乗になる。

つまり式で書くと

$$a^2 + b^2 = c^2$$

ここで

- **a と b**… 直角をはさむ 2 つの辺（「短いほうの辺」）
- **c**… 斜辺（直角の向かい側にある一番長い辺）

なぜそうなるのか（イメージで理解）

三平方の定理は「面積」で考えると分かりやすい。

1. 三角形の 3 つの辺に正方形をくっつける
2. 直角をはさむ 2 つの辺につく正方形の面積を足す
3. すると、斜辺につく正方形の面積とぴったり同じになる

つまり、長さの関係ではなく、面積の関係としても成り立っているということ。

具体例で見るともっと分かる

例えば、直角三角形の 2 辺が

- $a = 3$
- $b = 4$ だったら、

$$3^2 + 4^2 = 9 + 16 = 25$$

だから斜辺 c は

$$\sqrt{25} = 5$$

この「3・4・5」の組み合わせは有名な直角三角形だね。

 **どんなときに使う？**

- 三角形の辺の長さを求めたいとき
- 図形の距離を計算したいとき
- ゲームやプログラミングで「2点間の距離」を求めるとき
- 建築や設計で直角を扱うとき

数学だけでなく、いろんな場面で役立つ定理なんだ。

5. プロンプトエンジニアリングを行った場合

プロンプト：あなたは数学の博士です。三平方の定理を知らない人に説明するために、日本語で簡潔に三平の定理を説明してください。

AI からの回答：

三平方の定理は、**直角三角形の3つの辺の長さの関係を表すルール**です。

結論だけ先に言うと： 直角をはさんでいる2つの辺の長さをそれぞれ a 、 b 、直角の向かい側のいちばん長い辺（＝斜辺）を c とすると、

$$a^2 + b^2 = c^2$$

という関係が必ず成り立ちます。

 **もっと直感的に言うと**

- 四角形の「面積」を使って考えるとわかりやすいです。
- 辺 a の上に正方形を作ると面積は a^2 。
- 辺 b の上に正方形を作ると面積は b^2 。
- この2つの面積を足したものが、斜辺 c の上に作った正方形の面積 c^2 とぴったり同じになる、という不思議で美しい法則です。

💡 なぜ大事なのか

- 図形の長さを求めるときの基本中の基本
- 建築、測量、物理、プログラミングなど、いろんな場面で使われる
- 「距離」を計算するときの根本的な考え方になる

6. まとめ

AI から得られた二つの回答を比べるても、情報の精度や分かりやすに大きな違いが見られなかった。このことから簡単な質問の場合はプロンプトエンジニアリングをしなくても大丈夫なことが分かった。また、AI 技術の進歩に驚いた。しかし、今後もっと複雑な問題を解決するときに AI を用いる場合があると思うがその時はプロンプトエンジニアリングを行うことで AI の回答精度を上げていきたいと思った。

7. 最後に

このような未熟な文章を最後まで読んでくださりありがとうございました。

ローカルリポジトリを複数台で同期したい件

著者：lime

みなさんは、複数台の PC でソフトウェア開発をしていますか。複数台³⁰で開発している方はこんなことを思った（断定）。

「プロジェクトのソースコードを複数台で同期させたい」

... まあ、Git を使えば大体解決します。ただ、Git 管理をしていても、コミットにまとめたり、リモートリポジトリにプッシュしたりしないと同期することはできません。また、複数人で開発している場合には特に雑なコミット、雑なプッシュはできません。

そんなあなたに紹介するプロダクトがこちらの「Git ローカル同期システム」です。

※ここまで読んでいる人がどれだけいるかわかりませんが、ここまでの文章で分かるように、こちらの記事は Git を用いてソフトウェア開発をしている開発者向けのものとなっております。本当は Git について初学者の方にも理解しやすいものを書きたいと思っていましたが、本記事で紹介するシステムが完成して気分がはしゃいでしまっているため、仕方ないですね（ごめんなさい）。

さて、本システムの目的は、

「コミットできない変更分を別端末と同期させる」

です。加えて「メインのリモートリポジトリにプッシュできない/したくない状態の変更分」が対象です。メインのリモートにプッシュできるならそれが一番よいです。

（次のページに続く）

目的

プロジェクトのリモートリポジトリにプッシュできない変更分を、別端末と同期させる。

機能

- コミットしていない変更分（未コミットな変更）やスタッシュ（Git 標準機能で、変更分を一時的に退避させるもの）を別のローカルリポジトリに共有させる
- 未コミットな変更やスタッシュを別途 Git 管理することで、二重のバックアップを取ることができる
- プロジェクトのリモートリポジトリにはプッシュできない変更分を、別のリモートリポジトリにプッシュすることで解決
- 別リモートリポジトリへのプッシュ時に、プロジェクトのコミット内容を省くことで、同期用リモートリポジトリの容量を抑えることができる

原理

- Git の `worktree` 機能を利用して、プロジェクトの作業ディレクトリを変更せずに、別のブランチ操作を行う
- Git の `include.path` 機能を利用して、プロジェクトのローカルリポジトリにカスタムコマンドを追加する
- Git のリモートリポジトリを複数設定することで、プロジェクトのリモートリポジトリと同期用リモートリポジトリを分ける
- `git stash export` および `git stash import` を利用して、スタッシュの内容を別のローカルリポジトリに共有させる

同期の実行

1. 対象のプロジェクトのルートディレクトリをターミナル等で開く
2. 以下のコマンドを実行する

```
git sync
```

3. 同期されたチェックアウト状態と現在のローカルリポジトリのチェックアウト状態が同じであれば、未コミットな変更はそのままマージされる（でなければスタッシュに残る）

バックアップの実行

1. 対象のプロジェクトのルートディレクトリをターミナル等で開く
2. 以下のコマンドを実行する

```
git backup
```

実際に使ってみて

本システムでは作業ディレクトリをむやみに変更しないよう、メインのリモートリポジトリのプルは別途行う必要があります。そのため、`git pull`と`git sync`をそれぞれ行う必要があった点が少し面倒でした。また、未コミットな変更やスタッシュを管理するブランチの操作をメインのローカルリポジトリで行うと、作業ディレクトリが大きく変更されてしまうため、`worktree` 機能を利用しました。しかし、この機能はそれほど一般的ではないため、導入には手間がかかりました。

最後に

ここまで読んでくださりありがとうございます。詳しい説明もなく、たらたらと書き進めてしまいましたが、こんなシステムあっていいよなと思っていただけであれば幸いです。普段使っているツールも、自分が使いやすいようにカスタマイズするのってとても楽しくて、その楽しさが何となくでも伝わっていると嬉しいです。ソースコードは

<https://github.com/slerk3357/git-sync>

に公開していますので、気になった人は見てください。ソースコードは次のページ以降にも載せてあります。



35

```
1 *.sh text eol=lf
2 stash.gitconfig text merge=union
3 stash.bundle binary merge=take-theirs
```

リスト 1 .gitattributes ファイル

```
1 [merge "take-theirs"]
2   name = Always take theirs for binary files
3   driver = cp -f %B %A
4
5 [alias]
6   sync = "!f(){ ./git-sync/stash/sync.sh; }; f"
7   backup = "!f(){ ./git-sync/stash/backup.sh; }; f"
```

リスト 2 .gitconfig ファイル

```
1 [gitSync]
2   rootId = 73c9bab443d1f88ac61aa533d2eeaaa15451239c
3   stashTreeId = 4b825dc642cb6eb9a060e54bf8d69288fbee4904
4   checkoutBranch =
5   detachId =
```

リスト 3 stash.gitconfig ファイル


```
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  manage_stash_dir="./git-sync/stash"
5  stash_config_path="stash.gitconfig"
6
7  record_worktree_state() {
8      local branch_name detach_id dirty_state
9
10     branch_name=$(git symbolic-ref --quiet --short HEAD 2>/dev/null ||
true)
11     detach_id=$(git rev-parse HEAD)
12
13     if [ -n "$branch_name" ]; then
14         git config -f "$manage_stash_dir/$stash_config_path"
gitSync.checkoutBranch "$branch_name"
15     else
16         git config -f "$manage_stash_dir/$stash_config_path" --unset-all
gitSync.checkoutBranch 2>/dev/null || true
17     fi
18     git config -f "$manage_stash_dir/$stash_config_path" gitSync.detachId
"$detach_id"
19 }
20
21 echo "[backup] Initiating sync before backup..."
22 git sync
23
24 record_worktree_state
25
26 echo "[backup] Saving current work to stash..."
27 touch tmp_git_sync
28 git add .
29 popping_stash=$(git stash create "Stash uncommitted changes" || true)
30 git stash store -m "Stash uncommitted changes" "$popping_stash"
31 rm tmp_git_sync
32 git restore --staged tmp_git_sync || true
33
34 cd "$manage_stash_dir"
35
36 git stash export --to-ref refs/heads/bundled-stash
37
```

リスト 4 backup.sh ファイル (1/2)

```
37 38 echo "[backup] Bundling stashes..."
39 rm -f stash.bundle || true
40 git bundle create stash.bundle refs/heads/bundled-stash --not --
    exclude=refs/heads/bundled-stash --exclude=refs/stash --exclude=HEAD --
    exclude=worktrees/*/HEAD --all
41 git branch -D bundled-stash
42
43 git add .
44 git commit -m "backup modified files at $(date +%Y-%m-%d_%H-%M-%S)"
45 git push -u sync manage-stash
46
47 echo "[backup] Backup and push complete."
```

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  manage_stash_dir="./git-sync/stash"
5  stash_config_path="stash.gitconfig"
6
7  reconstruct_stash_chain() {
8      local root_commit current_parent git_tree stash_body_id original_msg
9
10     root_commit=$(git config -f "$stash_config_path" --get
gitSync.rootId 2>/dev/null || true)
11     git_tree=$(git config -f "$stash_config_path" --get
gitSync.stashTreeId 2>/dev/null || true)
12
13     current_parent="$root_commit"
14
15     while read -r stash_body_id; do
16         [ -n "$stash_body_id" ] || continue
17
18         original_msg="git stash: "$(git log -1 --format="%B"
"$stash_body_id")
19         export GIT_AUTHOR_NAME=$(git log -1 --format="%an"
"$stash_body_id")
20         export GIT_AUTHOR_EMAIL=$(git log -1 --format="%ae"
"$stash_body_id")
21         export GIT_AUTHOR_DATE=$(git log -1 --format="%aI"
"$stash_body_id")
22         export GIT_COMMITTER_NAME=$(git log -1 --format="%cn"
"$stash_body_id")
23         export GIT_COMMITTER_EMAIL=$(git log -1 --format="%ce"
"$stash_body_id")
24         export GIT_COMMITTER_DATE=$(git log -1 --format="%cI"
"$stash_body_id")
25
26         current_parent=$(printf "%s" "$original_msg" | git commit-tree -
p "$current_parent" -p "$stash_body_id" "$git_tree")
27         done < <(git config -f "$stash_config_path" --get-all
gitSync.stashId 2>/dev/null || true)
28
29     unset GIT_AUTHOR_NAME GIT_AUTHOR_EMAIL GIT_AUTHOR_DATE
GIT_COMMITTER_NAME GIT_COMMITTER_EMAIL GIT_COMMITTER_DATE

```

39

```
30     git stash import "$current_parent" >/dev/null 2>&1 || true
31 }
32
33
34 echo "[sync] Starting stash synchronization..."
35
36 cd "$manage_stash_dir"
37
38 git stash export --to-ref refs/heads/export-stash 2>/dev/null || true
39
40 git config -f "$stash_config_path" --unset-all gitSync.stashId
41 2>/dev/null || true
42 while read -r stash_id; do
43     [ -n "$stash_id" ] || continue
44     git config -f "$stash_config_path" --add gitSync.stashId "$stash_id"
45 done <<(git log --reverse --first-parent --format='%P' export-stash
46 2>/dev/null | awk '{ print $2 }' || true)
47 # export-stash の first-parent を辿って、各スタッシュチェーンコミットの
48 2 番目の親である本体コミットのみを記録する
49
50 if ! git ls-remote --exit-code --heads sync manage-stash >/dev/null
51 2>&1; then
52     echo "[sync] Remote branch sync/manage-stash does not exist yet."
53     git branch -D export-stash 2>/dev/null || true
54     cd "../.."
55     exit 0
56 fi
57
58 git stash clear
59 git add .
60 config_stashed=false
61 if git status --porcelain; then
62     git stash push -m "Backup stash config"
63     config_stashed=true
64 fi
65
66 git fetch sync manage-stash
67 git merge --ff-only manage-stash
68
69 git fetch ../stash.bundle bundled-stash:bundled-stash
70
```

```

67 popping_stash=$(git rev-parse refs/heads/bundled-stash^2)
68 git branch -f bundled-stash refs/heads/bundled-stash~1
69
70 if [ "$config_stashed" = true ]; then
71     git stash pop
72 fi
73
74 reconstruct_stash_chain
75
76 git branch -D export-stash bundled-stash 2>/dev/null || true
77
78 if [ -n "$popping_stash" ]; then
79     git stash store -m "stash to pop" "$popping_stash"
80 fi
81
82 cd "../.."
83
84 STATUS=0
85 current_branch=$(git symbolic-ref --quiet --short HEAD 2>/dev/null ||
true)
86 if [ -n "$current_branch" ]; then
87     if [ "$current_branch" = "$(git config -f
"$manage_stash_dir/$stash_config_path" --get gitSync.checkoutBranch
2>/dev/null || true)" ]; then
88         echo "[sync] Popping stash..."
89         git add .
90         git stash pop || STATUS=$?
91     else
92         echo "[sync] Current branch does not match the recorded branch.
Skipping stash pop."
93     fi
94 elif [ "$(git rev-parse HEAD)" = "$(git config -f
"$manage_stash_dir/$stash_config_path" --get gitSync.detachId
2>/dev/null || true)" ]; then
95     echo "[sync] Popping stash..."
96     git add .
97     git stash pop || STATUS=$?
98 else
99     echo "[sync] Current detached HEAD does not match the recorded
detached commit. Skipping stash pop."
100 fi

```

```
41 101  
    102 rm tmp_git_sync  
    103 git restore --staged tmp_git_sync 2>/dev/null || true  
    104  
    105 exit $STATUS
```

あとがき

まず、今年度も部誌を出すことができたことに心より感謝の意を示したいと思います。今年度も複数の記事の提供をいただき、この部誌をよい交流の場にできたと考えております。製作に協力してくださった部員の方々、部誌のページを開きここまで読んでくださった読者の方々、本当にありがとうございました。

今年度から、部の活動形態をガラッと変えて「チームを半期ごとに構成しなおす」活動にしました。部のチームの活動の停滞を回避し、部員間のコミュニケーションを強化する目的で実施しました。前期は、CGアニメーションを作ったり、リズムゲームを作ったり、講習会を運営したりする合計5つのチームが構成されました。うまくいったチームもあれば、前期の最後の方では停滞気味だったチームもあり、改善が必要そうです。後期には新しくチームを編成し、活動するので前期の経験を活かし進めていきたいと考えています。

部誌の製作に協力してくださった方々、日々の電算部の活動を支えてくださっている方々、本当にありがとうございます。(2026年度 部長 nirocon)

D言語 2026年度版

2026年9月4日 初版

著者 : Nanmo, ZIMMAD, ゆーてぃー, Shimpei Moribe, lime, nirocon
校閲 : ゆーてぃー
表紙 : 間野谷/m-nagano
編集発行 : 群馬高専 電算部
X(Twitter) : @GNCT_densan

誤字脱字等ありましたら、X(@GNCT_densan) までご連絡ください。

©2026 National Institute of Technology, Gunma College Densan Club.